Day 4 - LinkedList, Stack, Queue Interview Notes

Day 4 — LinkedList, Stack, Queue: Interview-Ready Notes

Day 4 — LinkedList, Stack, Queue: Interview-Ready Notes

(Code examples in Java. Concepts apply to any language.)

PART 1: LINKED LIST

🧠 1.1 Internal Working

A Linked List is a **linear data structure** where elements (“nodes”) are NOT stored in contiguous memory. Each node holds: - **data** - **pointer/reference** to the next node (and previous, for doubly linked lists)

```
[data|next] -> [data|next] -> [data|next] -> null
```

Why it matters internally: - Array: contiguous memory, O(1) random access, but O(n) insertion/deletion (shifting). - LinkedList: scattered memory (heap-allocated nodes), O(n) access, but O(1) insertion/deletion **once you have the pointer**. - No pre-allocation needed — grows dynamically, one node at a time, unlike arrays which may need resizing (copy to new array). - Extra memory overhead per node (the pointer(s)) — this is the trade-off for flexibility. - Poor cache locality (nodes scattered in heap) vs arrays (cache-friendly, sequential).

Types: | Type | Structure | |—|—| | Singly Linked List | node -> next | | Doubly Linked List | prev <- node -> next | | Circular Linked List | last node points back to head | | Circular Doubly Linked List | both directions circular |

💼 1.2 Real-World Examples

- **Browser history (back/forward)** — doubly linked list.
- **Music/video playlist “next/previous”** navigation.
- **Undo/Redo** in text editors (often stack + linked list based).
- **Image viewer** (next/prev photo).
- **Memory management** — OS free-block lists are linked lists.
- **Hash Map collision handling** (separate chaining uses linked lists per bucket).
- **Blockchain** — each block points to the previous block’s hash (conceptually a linked list).

💻 1.3 Complete Code — Singly Linked List (Java)

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; this.next = null; }
}

class LinkedList {
    Node head;

    // Insert at end - O(n)
    void insertEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) { head = newNode; return; }
        Node curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = newNode;
    }

    // Insert at beginning - O(1)
    void insertBegin(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Delete a node by value - O(n)
    void delete(int key) {
        if (head == null) return;
        if (head.data == key) { head = head.next; return; }
        Node curr = head;
        while (curr.next != null && curr.next.data != key) curr = curr.next;
        if (curr.next != null) curr.next = curr.next.next;
    }

    // Search - O(n)
    boolean search(int key) {
        Node curr = head;
        while (curr != null) {
            if (curr.data == key) return true;
            curr = curr.next;
        }
        return false;
    }

    // Reverse the list - O(n), O(1) space
    void reverse() {
        Node prev = null, curr = head, next = null;
        while (curr != null) {
            next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        head = prev;
    }

    void print() {
        Node curr = head;
        while (curr != null) {
            System.out.print(curr.data + " -> ");
            curr = curr.next;
        }
        System.out.println("null");
    }
}
```

💻 1.4 Doubly Linked List (Java)

```java
class DNode {
    int data;
    DNode prev, next;
    DNode(int data) { this.data = data; }
}

class DoublyLinkedList {
    DNode head, tail;

    void insertEnd(int data) {
        DNode node = new DNode(data);
        if (head == null) { head = tail = node; return; }
        tail.next = node;
        node.prev = tail;
        tail = node;
    }

    void deleteNode(DNode node) {
        if (node.prev != null) node.prev.next = node.next;
        else head = node.next;
        if (node.next != null) node.next.prev = node.prev;
        else tail = node.prev;
    }
}
```

PART 2: STACK

🧠 2.1 Internal Working

LIFO — Last In, First Out. Think of a stack of plates: you add/remove from the **top only**.

Internal implementations: 1. **Array-based**: fixed/dynamic array + a `top` pointer. Push = `arr[++top] = x`. Pop = `arr[top--]`. O(1) amortized. 2. **Linked-list based**: push/pop at the **head** — O(1), no resizing needed, but extra pointer memory per node.

Key internal detail: Function call stack (recursion) in every programming language is a literal stack — each function call pushes a "stack frame" (local vars, return address); returning pops it. This is *why* deep recursion causes **StackOverflowError**.

💼 2.2 Real-World Examples

- **Undo/Redo** functionality (Ctrl+Z).
- **Browser back button** (page history stack).
- **Function call stack / recursion** (compilers, interpreters).
- **Expression evaluation** (compilers use stack to evaluate `(3+4)*2`).
- **Balanced parentheses/syntax checking** (IDEs, compilers).
- **Backtracking algorithms** (maze solving, N-Queens).

💻 2.3 Complete Code — Stack (Java)

Array-based:

```
class ArrayStack {
    int[] arr;
    int top;
    int capacity;

    ArrayStack(int size) {
        arr = new int[size];
        capacity = size;
        top = -1;
    }

    void push(int x) {
        if (top == capacity - 1) throw new RuntimeException("Stack Overflow");
        arr[++top] = x;
    }

    int pop() {
        if (top == -1) throw new RuntimeException("Stack Underflow");
        return arr[top--];
    }

    int peek() {
        if (top == -1) throw new RuntimeException("Stack is empty");
        return arr[top];
    }

    boolean isEmpty() { return top == -1; }
}
```

Linked-list based:

```java
class StackLL {
    Node top;
    class Node {
        int data;
        Node next;
        Node(int data) { this.data = data; }
    }

    void push(int x) {
        Node node = new Node(x);
        node.next = top;
        top = node;
    }

    int pop() {
        if (top == null) throw new RuntimeException("Stack Underflow");
        int val = top.data;
        top = top.next;
        return val;
    }

    int peek() { return top.data; }
    boolean isEmpty() { return top == null; }
}
```

Using Java’s built-in:

```java
import java.util.Stack;
Stack<Integer> stack = new Stack<>();  // or use Deque (recommended over Stack class)
Deque<Integer> stack2 = new ArrayDeque<>();
stack2.push(10); stack2.pop(); stack2.peek();
```

PART 3: QUEUE

🧠 3.1 Internal Working

FIFO — First In, First Out. Add at **rear**, remove from **front**.

Internal implementations: 1. **Array-based (circular)**: uses `front` and `rear` pointers with **modulo arithmetic** to wrap around and avoid wasting space (a naive array queue wastes slots after dequeues). 2. **Linked-list based**: pointers to `head` (front) and `tail` (rear) — O(1) enqueue/dequeue. 3. **Two-stack based Queue**: simulate FIFO using two LIFO stacks (classic interview trick).

Circular Queue key formula:

```
rear = (rear + 1) % capacity
front = (front + 1) % capacity
isFull  = (rear + 1) % capacity == front
isEmpty = front == -1 (or count == 0)
```

Variants: | Type | Behavior | |—|—| | Simple Queue | FIFO only | | Circular Queue | reuses freed space | | Deque (Double-ended) | insert/delete both ends | | Priority Queue | dequeues by priority, not order (usually heap-based) |

💼 3.2 Real-World Examples

- **Print spooler** (jobs printed in order received).
- **CPU task scheduling** (round-robin scheduling).
- **Customer service / call center systems** (first come, first served).
- **BFS traversal** in graphs/trees.
- **Message queues** (Kafka, RabbitMQ — real production systems).
- **Ticket booking systems** (queue-based fairness).

💻 3.3 Complete Code — Queue (Java)

Circular array-based:

```java
class CircularQueue {
    int[] arr;
    int front, rear, size, capacity;

    CircularQueue(int cap) {
        capacity = cap;
        arr = new int[cap];
        front = 0; rear = -1; size = 0;
    }

    void enqueue(int x) {
        if (size == capacity) throw new RuntimeException("Queue Full");
        rear = (rear + 1) % capacity;
        arr[rear] = x;
        size++;
    }

    int dequeue() {
        if (size == 0) throw new RuntimeException("Queue Empty");
        int val = arr[front];
        front = (front + 1) % capacity;
        size--;
        return val;
    }

    int peek() { return arr[front]; }
    boolean isEmpty() { return size == 0; }
}
```

Linked-list based:

```java
class QueueLL {
    class Node {
        int data;
        Node next;
        Node(int data) { this.data = data; }
    }
    Node front, rear;

    void enqueue(int x) {
        Node node = new Node(x);
        if (rear == null) { front = rear = node; return; }
        rear.next = node;
        rear = node;
    }

    int dequeue() {
        if (front == null) throw new RuntimeException("Queue Empty");
        int val = front.data;
        front = front.next;
        if (front == null) rear = null;
        return val;
    }
}
```

Queue using two Stacks (classic problem):

```java
class QueueUsingStacks {
    Stack<Integer> s1 = new Stack<>(); // for enqueue
    Stack<Integer> s2 = new Stack<>(); // for dequeue

    void enqueue(int x) { s1.push(x); }

    int dequeue() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) s2.push(s1.pop());
        }
        return s2.pop();
    }
}
```

? PART 4: 35 INTERVIEW QUESTIONS (Conceptual)

LinkedList

1. What is a linked list and how does it differ from an array?
2. Singly vs doubly vs circular linked list — differences?
3. Time complexity of insertion at head vs tail vs middle?
4. Why is random access O(n) in a linked list?
5. How do you detect a cycle in a linked list? (Floyd's algorithm)
6. How do you find the middle of a linked list in one pass?
7. How do you reverse a linked list iteratively and recursively?
8. What is a "dummy node" and why is it used?
9. How is a LinkedList used internally in Java's `LinkedHashMap` or `HashMap` buckets?
10. What are the memory trade-offs of linked list vs array?
11. How do you merge two sorted linked lists?
12. What happens if you don't nullify the `next` pointer of the last node after deletion — memory leak?
13. Difference between shallow copy and deep copy of a linked list with random pointers?

Stack

14. What is a stack? Real-life analogy?
15. How is recursion related to the stack data structure?
16. What causes a `StackOverflowError`?
17. How do you implement a stack using an array vs linked list — pros/cons?
18. How do you implement a stack using two queues?
19. How do you check for balanced parentheses using a stack?
20. What is the "Next Greater Element" problem and how is a stack used?
21. Explain how the "Min Stack" (O(1) getMin) is implemented.
22. What is the time complexity of push/pop/peek?
23. How is a stack used in the "Undo" feature of an application?
24. Explain infix to postfix conversion using a stack.

Queue

25. What is a queue? How is it different from a stack?
26. Why do we need a circular queue instead of a simple array queue?
27. How do you implement a queue using two stacks?
28. What is a priority queue and how is it typically implemented internally (heap)?
29. What is a deque, and where is it used?
30. How is a queue used in BFS traversal?
31. What is the difference between `poll()` and `remove()` in Java's `Queue` interface?
32. Explain round-robin CPU scheduling using a queue.
33. How would you design a queue that also supports retrieving the max element in O(1)?
34. What's the time complexity of enqueue/dequeue in a circular queue vs a naive array queue?
35. How does a blocking queue work in multithreading (producer-consumer problem)?

Cross-cutting

36. When would you choose a LinkedList over an ArrayList in Java?
37. How does Java's `ArrayDeque` internally work, and why is it preferred over `Stack`/`Vector`?
38. What's the space complexity difference between array-based and linked-list-based implementations?
39. How would you make a stack/queue thread-safe?
40. Explain amortized O(1) in the context of dynamic array resizing.

🧩 PART 5: CODING PROBLEMS (Practice List)

LinkedList

1. Reverse a linked list (iterative + recursive).
2. Detect and remove a cycle in a linked list (Floyd's Tortoise & Hare).
3. Find the middle of a linked list.
4. Merge two sorted linked lists.
5. Remove Nth node from the end (two-pointer technique).
6. Check if a linked list is a palindrome.
7. Add two numbers represented as linked lists.
8. Clone a linked list with random pointers.
9. Detect intersection point of two linked lists.
10. Flatten a multilevel linked list.

Stack

11. Valid Parentheses (balanced brackets).
12. Min Stack — design O(1) getMin().
13. Next Greater Element (using monotonic stack).
14. Evaluate Reverse Polish Notation (postfix expression).
15. Implement a stack using queues.
16. Largest Rectangle in Histogram (monotonic stack).
17. Sort a stack using recursion only.
18. Daily Temperatures problem (monotonic stack).
19. Design a stack that supports `getMax()` in O(1).
20. Simplify Unix-style file path (`/a/./b/../../c/`).

Queue

21. Implement a queue using stacks.
22. Design a circular queue.
23. Sliding window maximum (using deque).
24. Implement a queue that supports `getMax()` in O(1) (monotonic deque).
25. First non-repeating character in a stream (queue-based).
26. Design a Least Recently Used (LRU) Cache (deque + hashmap).
27. Rotten Oranges problem (BFS using queue).
28. Generate binary numbers from 1 to N using a queue.
29. Implement a circular tour / gas station problem (queue logic).
30. Task Scheduler (priority queue based).

🤖 PART 6: AI-STYLE / SCENARIO-BASED INTERVIEW QUESTIONS

These test *reasoning* and *design thinking*, common in modern (AI-augmented / system-design-flavored) interviews:

1. **“Design a system where you need O(1) insertion, deletion AND random access — is a linked list or array better, and why? Could a hybrid structure help?”** *(Expected insight: neither alone is ideal — discuss skip lists, or hash map + doubly linked list combo, as used in LRU Cache.)*
2. **“You’re asked to implement Undo/Redo for a text editor with unlimited history but low memory. How would you design it?”** *(Expected: two stacks — undo stack & redo stack; or a doubly linked list acting as a “cursor” over history.)*
3. **“A queue-based system (like a ticket booking service) needs fairness AND priority for VIP users. How do you redesign the queue?”** *(Expected: multiple queues by priority tier, or a priority queue/heap with tie-breaking by arrival time.)*
4. **“Your linked list based cache is O(n) for lookups. How do you make lookups O(1) while keeping O(1) eviction order?”** *(Expected: combine HashMap + Doubly Linked List → this IS the LRU cache design.)*
5. **“If asked to reverse a linked list without extra space and you can only traverse once — walk me through your thinking out loud.”** *(Tests articulation of the 3-pointer (prev, curr, next) technique.)*
6. **“How would you detect if two linked lists merge at some point, without using extra memory?”** *(Expected: two-pointer length-difference technique, or Floyd’s cycle-style pointer reset trick.)*
7. **“Explain trade-offs: would you use recursion (implicit stack) or an explicit stack for tree traversal in a memory-constrained embedded system?”** *(Tests understanding of call-stack overhead vs explicit stack control.)*
8. **“How would a message queue system (like Kafka) differ internally from your basic Queue implementation, and why can’t a simple linked-list queue scale to millions of messages/sec?”** *(Expected: discuss persistence, partitioning, disk-backed logs vs in-memory pointers, multi-consumer offsets.)*
9. **“If your interviewer says ‘don’t use built-in Stack/Queue classes,’ how do you justify picking array vs linked list for your custom implementation given the constraints (fixed max size vs unknown size)?”**
10. **“Explain how you’d design a browser’s back/forward navigation system, including handling a new page visit after going back (redo history should be cleared).”** *(Expected: two stacks, and clearing the “forward” stack on new navigation — a very common real design question.)*

🎤 PART 7: MOCK INTERVIEW (Sample Q&A Flow)

Interviewer: Let’s start simple. What’s the difference between an array and a linked list?

You: An array stores elements in contiguous memory, giving O(1) random access but O(n) insertion/deletion because of shifting. A linked list stores elements as nodes connected via pointers — O(n) access since we must traverse, but O(1) insertion/deletion once we have a reference to the node, since we just re-link pointers.

Interviewer: Good. Now, how would you detect a cycle in a linked list?

You: I’d use Floyd’s Cycle Detection (tortoise and hare). Two pointers — slow moves one step, fast moves two steps. If there’s a cycle, they will eventually meet inside the loop. If fast reaches null, there’s no cycle. It’s O(n) time, O(1) space — better than using a HashSet which needs O(n) space.

Interviewer: Nice. Can you code that quickly?

You:

```
boolean hasCycle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Interviewer: Now switch gears — implement a stack such that you can retrieve the minimum element in O(1).

You: I’ll maintain two stacks — the main stack, and an auxiliary “min stack” that tracks the current minimum at each level. When I push `x`, I also push `min(x, currentMin)` onto the min-stack. When I pop, I pop from both. `getMin()` just peeks the min-stack top. Both operations stay O(1) time, with O(n) extra space.

Interviewer: Why not just track a single `min` variable instead of a whole second stack?

You: Because if I pop the current minimum element off the stack, I need to know what the *previous* minimum was — a single variable would lose that history. The min-stack preserves the minimum at every state, so popping restores the correct earlier minimum automatically.

Interviewer: Great answer. Last one — design a queue using two stacks.

You: I’ll use `s1` for enqueue and `s2` for dequeue. On enqueue, just push to `s1` — O(1). On dequeue, if `s2` is empty, I pour all elements from `s1` into `s2` (which reverses their order, making the oldest element land on top of `s2`), then pop from `s2`. This gives amortized O(1) dequeue, since each element moves from `s1` to `s2` exactly once over its lifetime.

Interviewer: Solid. That’s all from my side.

📌 Quick Revision Cheat Sheet

Structure	Access	Insert (head)	Insert (tail)	Delete	Search
Array	O(1)	O(n)	O(1)*	O(n)	O(n)
Singly Linked List	O(n)	O(1)	O(n)/O(1) w/ tail ptr	O(n)	O(n)
Doubly Linked List	O(n)	O(1)	O(1)	O(1)**	O(n)
Stack (array/LL)	O(n)	—	O(1) push	O(1) pop	O(n)
Queue (circular/LL)	O(n)	—	O(1) enqueue	O(1) dequeue	O(n)

* amortized, due to occasional resizing ** O(1) if you already have the node reference

---

Good luck with Day 4! Next suggested topics: Trees (BST, traversals), HashMap internals, and Heaps/Priority Queue deep-dive.